

AUTOMATED VULNERABILITY DISCOVERY

What are vulnerabilities?

From unintended behavior to attacker capability

Prof. Matt Revelle
Montana State University

What vulnerabilities allow

Out-of-bounds read

Leak passwords, keys,
and pointers from
memory

Out-of-bounds write & Buffer overflow

Corrupt data/values in
memory

Integer overflow

Unexpected state that
can lead to other
capabilities

Path traversal

Leak credentials, private
files

Command injection

Run attacker-chosen
commands

Use after free

Leak sensitive
information, hijack
control flow

Where security problems come from

DESIGN FLAW

The intended behavior is wrong

Example: "Encrypt" important data with ROT13.

IMPLEMENTATION BUG

The implementation of intended behavior is wrong

Example: The spec says

$x \geq 10$; the code checks

$x > 0$.

Design flaw: account recovery bypasses MFA

The normal login is strong; the intentionally weaker recovery path defeats it.

SYSTEM GOAL

Only the account owner can regain access

Normal login requires both a password and a registered device.

CHOSEN DESIGN

Support may reset MFA using name and date of birth

The recovery policy deliberately accepts weaker evidence.

FAILURE

An attacker supplies public information

The system faithfully follows its policy and transfers the account.

The design of the recovery policy is insecure.

Design flaw: firmware updates are not authenticated

The updater checks whether a package looks valid, but not who authorized it.

SYSTEM GOAL

Only vendor-approved firmware may run

An update must preserve control of the device's software.

CHOSEN DESIGN

Install any well-formed `update.bin` from USB

The format is checked, but there is no cryptographic signature.

FAILURE

A malicious package passes every designed check

The updater correctly installs attacker-provided firmware.

The missing check is in the design, not the code.

Implementation bug: the authorization check is too weak

```
def read_report(user, report):  
    if not user.is_authenticated:  
        raise AccessDenied()  
    return report.contents
```

INTENDED RULE

Only the report owner or an administrator may read it

ACTUAL CHECK

Any logged-in user is accepted

Changing the report identifier exposes another user's report.

The rule is adequate; this code fails to enforce it.

Implementation bug: one character changes the boundary

```
void process_batch
(Item *items,
 size_t count)
{
    for (size_t i = 0;
        i <= count;
        i++)
        process(items[i]);
}
```

INTENDED RULE

Process exactly **count** items

Valid indices run from **0** through **count - 1**.

ONE-CHARACTER TYPO

i <= count permits **i == count**

The final iteration accesses one item beyond the array.

Changing **<=** to **<** restores the intended boundary.

Why implementations diverge from the design

Incorrect code can begin with misunderstanding or a simple mechanical mistake.

SPECIFICATION

The developer does not understand the specification

They implement a different rule than the design intended.

MENTAL MODEL

The developer does not understand what the code does

The semantics of the language or library differ than developer's mental model.

MECHANICAL ERROR

The developer makes a typo or copy/paste mistake

A wrong variable name, duplicated condition, etc. changes the behavior.

An adequate design does not guarantee that it will be understood or built correctly.

Why implementations diverge from the design (cont.)

A developer can understand the intended rule and still miss where or when it must hold.

OMISSION

An edge case or state is overlooked

The common path works, but a less common path is missed.

INCOMPLETE CHANGE

Only some code paths are updated

Code is only partially updated with a bug fix or new feature.

Implementation bugs are often mundane.

Is this a bug?

```
y = y / x
```

What must we know before we can answer?

- Can `x` be zero?
- What does the language do?
- Is failure acceptable?
- What does the caller expect?

How about this?

```
if x > 0:  
    y = y / x
```

Safety property

Zero cannot reach the division through this path.

Correctness question

Should negative values also be divided?

Is it a bug now?

```
# Divide y by x only  
# when x >= 10.  
if x > 0:  
    y = y / x
```

Which inputs expose
the bug?

$$1 \leq x < 10$$

When does a defect become a vulnerability?



Reachability

Can attacker-controlled input reach the code and establish its trigger conditions?

Utility

Does the effect advance confidentiality, integrity, availability, or privilege goals?

Vulnerability, PoC, and exploit

CLAIM

Vulnerability

A condition that can provide attacker-useful behavior.

PROOF

Proof of concept

Demonstrates the vulnerability can be exercised.

USE

Exploit

Exercises one or more vulnerabilities to achieve a goal.

A PoC can establish a vulnerability without being a complete exploit.

Exploits often chain multiple vulnerabilities

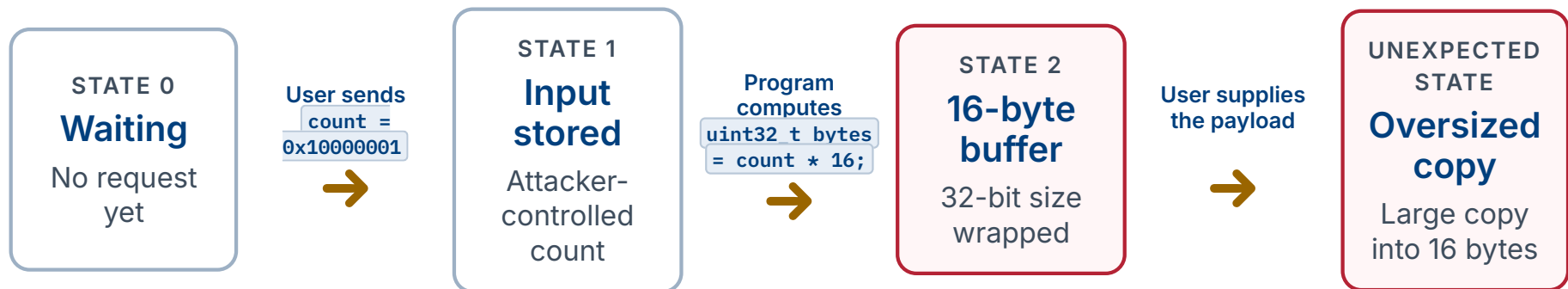


Each step creates a condition or capability that the next step needs.

An exploit commonly involves multiple steps.

User interactions drive state transitions

The user chooses **values and action order**; the program maps each interaction to a new state.



Attackers do not set memory directly; they steer the program into a reachable state the designer did not intend.

Vulnerability classes

Common ways things can go wrong

Integer overflow

```
void receive_records(  
    uint32_t count,  
    const char *input)  
{  
    uint32_t bytes =  
        count * (uint32_t)  
            sizeof(struct record);  
    struct record *records =  
        malloc(bytes);  
  
    if (records != NULL)  
        memcpy(records, input,  
            (size_t)count *  
            sizeof *records);  
    free(records);  
}
```

1 · Arithmetic wraps

`bytes` becomes smaller than the mathematical product.

2 · Allocation is undersized

The object is valid—but too small.

3 · Copy uses a larger size

The mismatch becomes a heap overflow.

Out-of-bounds access

```
int read_score(  
    const int scores[4],  
    unsigned i)  
{  
    if (i <= 4) return scores[i];  
    return -1;  
}
```

Out-of-bounds read

Index 4 may disclose adjacent data—or crash.

```
int update_score(  
    int scores[4],  
    unsigned i, int v)  
{  
    if (i <= 4) {  
        scores[i] = v;  
        return 0;  
    }  
    return -1;  
}
```

Out-of-bounds write

Index 4 may corrupt adjacent state.

Buffer overflow

Stack-based overflow

```
char name[16];  
strcpy(name, input);
```

Unchecked write past a stack object.

Heap-based overflow

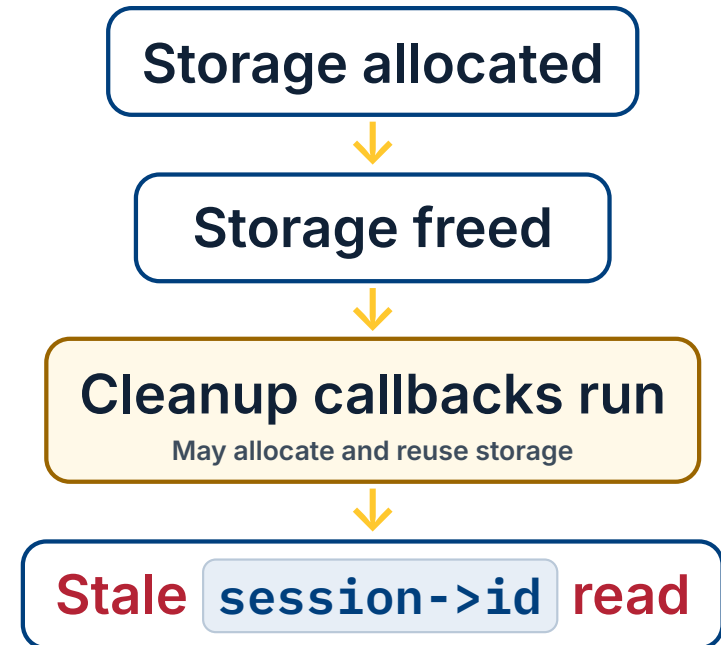
```
char *message = malloc(16);  
memcpy(message, input, length);
```

Attacker-controlled length exceeds a heap object.

The vulnerability is a buffer overflow—and stack or buffer distinguishes where the buffer is stored.

Use after free

```
void close_session(  
    struct session *session)  
{  
    unregister_session(session);  
    free(session);  
  
    run_cleanup_callbacks();  
    audit_log("closed session %u",  
             session->id);  
}
```



Command injection

```
import subprocess

def check_host(host):
    command = f"ping -c 1 {host}"
    return subprocess.run(
        command, shell=True)
```

ATTACKER INPUT

127.0.0.1; touch /tmp/injection-demo

SHELL RECEIVES

ping -c 1 127.0.0.1; touch
/tmp/injection-demo

The semicolon starts a second command

The shell runs `ping`, then creates `/tmp/injection-demo`.

Path traversal

```
import os

def read_report(name):
    path = os.path.join(
        "/srv/reports", name)
    with open(path, "rb") as report:
        return report.read()
```

ATTACKER INPUT

../../etc/passwd

JOINED PATH

/srv/reports/../../etc/passwd

FILESYSTEM RESOLVES IT TO

/etc/passwd

Containment failed

The program opens a file outside

/srv/reports.

Core definitions

Bug

A defect in an implementation that makes the system behave differently from its intended behavior.

Design flaw

An intended behavior or mechanism that fails to satisfy a relevant system goal, policy, or invariant.

Vulnerability

A reachable bug or design flaw whose effect can advance an attacker toward a security goal.

Proof of concept

A demonstration that the claimed behavior can occur.

Exploit

The use of one or more vulnerabilities to use a program to achieve some goal.

Vulnerabilities are the building blocks of exploits.

Questions?